MyCodeSchool

# Code review — sebastian-online-school

Runde de review pe proiectul tău, cea mai nouă sus.

---

## Runda 1 — 2026-10-20 — sha de3524b

Review pe `de3524b`, pornit pe H2 și exersat prin cereri reale (curl) pe căile semnalate. Fiecare 🔴 e reprodus, nu dedus, în afară de locurile unde scrie explicit „din cod". Rundele vechi sunt condensate la sfârșit.

**Rezumat:** 6 🔴, 10 🟡, 7 🟢. Cele mai grave: parola criptată a utilizatorului ajunge în JSON de mii de ori (B1) și orice student poate modifica orice profesor (B2).

---

## 🔴 Critice

### B1 🔴 Răspunsul de profesor conține entitatea `Course`: recursie infinită și hash-ul parolei în JSON

`src/main/java/ro/mycode/sebischool/professor/dtos/ProfessorResponse.java:14`

**CUM E ACUM:** `GET /api/v2/profesori` (după ce ai atribuit un curs) întoarce `200`, 502373 bytes, JSON trunchiat (se termină în `}}}}...`), iar hash-ul bcrypt al profesorului apare de 2110 ori. În log: `HttpMessageNotWritableException: Infinite recursion (StackOverflowError)` și `response committed already`.

**DE CE:** `ProfessorResponse` are câmpul `Set<Course> courses`, deci Jackson primește ENTITATEA `Course`, nu un DTO. Jackson serializează orice getter găsește: `Course.getProfessor()` -> `Professor.getUser()` -> `User` (cu `getPassword()`) -> `User.getProfessor()` -> `Professor.getCourses()` -> iar `Course` ... Relațiile bidirecționale formează un ciclu, iar Jackson nu are cum să știe unde să se oprească.

Lazy loading nu te salvează: `open-in-view` e activ implicit, deci sesiunea Hibernate e încă deschisă cât timp Jackson scrie, iar fiecare proxy lazy se încarcă la cerere.

De ce vezi `200` și nu `500`: Jackson a apucat să scrie deja ~500 KB în răspuns (statusul a plecat cu primii bytes) când a dat de `StackOverflowError`. De-aia scrie în log `response committed already`: clientul primește un JSON stricat, care conține parola criptată a utilizatorului de mii de ori.

**FIX:** `ProfessorResponse` și `ProfessorSummaryResponse` primesc `List<CourseSummaryResponse>` (sau un mic record `CourseRef(id, name)`), iar `ProfessorMapper` mapează `professor.getCourses().stream().map(CourseMapper::toDto)`. Regula de reținut: un controller nu întoarce niciodată o entitate JPA, nici măcar împachetată într-un DTO.

**Același bug la:**

- `professor/dtos/ProfessorSummaryResponse.java:15` - același `Set<Course> courses`. Nu e expus acum, pentru că `ProfessorMapper.toSummary` (`ProfessorMapper.java:17–34`) nu e apelat de niciun endpoint; dacă nu-l ștergi, bug-ul se repetă la prima folosire.
- `professor/mapper/ProfessorMapper.java:46` - `professor.getCourses()` pasat în `ProfessorResponse`: locul unde entitatea intră în DTO.
- `professor/mapper/ProfessorMapper.java:31` - același lucru pentru `ProfessorSummaryResponse` (cod mort acum).
- Toate endpoint-urile care întorc `ProfessorResponse` trec prin `toWithCourses` (`ProfessorCommandServiceImpl.java:53, 65, 95, 110`; `ProfessorQueryServiceImpl.java:24, 32`): `ProfessorController.java:44` (PUT `/{id}`), `:51` (PATCH `/{id}`), `:58` (GET `), :65 (POST /{id}/courses/{courseId}), :72 (DELETE /{id}/courses/{courseId}). Prin rulare am verificat doar GET și POST de atribuire; restul sunt confirmate din cod.` ProfessorQueryService.getProfessorById `(:10,` ProfessorQueryServiceImpl.java:32`) întoarce același tip, dar nu are endpoint.
- Alte DTO-uri (Course, Student, Enrolment, Book, Auth, User) nu importă clase din `*.model`, în afară de enum-ul `UserType` din `AuthRegisterRequest.java:7` (verificat doar prin grep, nu prin rulare).
- Risc asociat: `users/model/User.java:65` - `toString()` include `password`, iar `Professor.java:59` include `user` și `courses`. Nu ajunge în JSON, dar poate scurge hash-ul în log.

După fix, verifică: `GET /api/v2/profesori` nu mai conține `password` și `professor` în corp.

### B2 🔴 Un student poate modifica orice profesor

`src/main/java/ro/mycode/sebischool/professor/controller/ProfessorController.java:50`

**CUM E ACUM:** cu tokenul Anei (STUDENT), `PATCH /profesori/1 {"specialty":"Patched by Ana"}` -> `200`; `PUT /profesori/1 {..."specialty":"Hacked","departament":"Nimic"...}` -> `200`, iar profesorul a fost modificat.

**DE CE:** `@PreAuthorize` verifică un singur lucru: dacă string-ul cerut se află printre authorities din token. Nu știe nimic despre roluri sau despre cine e proprietarul resursei. În `AuthServiceImpl.permissionsForType` studentul primește `USER_VIEW` și `USER_EDIT`, exact ca profesorul. Pe PATCH ai pus `USER_VIEW`, o permisiune de CITIRE, deci orice utilizator logat poate scrie. PUT cere `USER_EDIT`, pe care tot studentul îl are. Rezultatul: permisiunea nu deosebește studentul de profesor, iar verificarea care ar fi trebuit să blocheze se face fără să blocheze pe nimeni.

**FIX:** PATCH primește `@PreAuthorize("hasAuthority('USER_EDIT')")`, iar studentul nu mai primește `USER_EDIT` în `permissionsForType` (sau creezi o permisiune separată, `PROFESSOR_EDIT`, pe care o are doar profesorul). Pentru resursele proprii ale studentului ai nevoie și de verificare de proprietar (id-ul din path = utilizatorul din token).

**Același bug la:**

- Cauza de fond: `auth/authService/AuthServiceImpl.java:144–150` (`permissionsForType`: STUDENT primește `USER_VIEW` + `USER_EDIT`; PROFESSOR le primește la `:136–142`, la fel de nediferențiate).
- `ProfessorController.java:42–44` PUT (`USER_EDIT`, îl are studentul) și `:56–58` GET `/profesori` (`USER_EDIT`: studentul listează toți profesorii).

- `StudentController.java:47–49` PUT `/students/{id}` : `USER_EDIT`, deci orice student modifică orice alt student.
- `StudentController.java:41–46` PATCH `/students/{id}` : fără `@PreAuthorize`.
- `CourseController.java:43–48` PATCH `/courses/{id}` : fără `@PreAuthorize`, un student modifică orice curs (PUT la `:49–51` cere corect `COURSE_MANAGE`, doar PATCH ocolește).
- `EnrolmentController.java:43–48` PATCH și `:49–54` PUT `/enrolments/{id}` : fără `@PreAuthorize`, orice autentificat modifică orice înscriere.
- `EnrolmentController.java:29–34` POST `/add` cu `ENROL_SELF` : nu verifică că `studentId` din request e al utilizatorului curent (înscrie alt student).
- GET-uri fără `@PreAuthorize` : `EnrolmentController.java:55–60, :61–65, :67–71` ; `StudentController.java:54–58, :61–65, :67–72` (orice autentificat citește datele oricărui student); `CourseController.java:63–67, :69–74` (GET all la `:56–58` are `COURSE_VIEW` : inconsistent).
- `BookController.java:28–74` : TOATE rutele (POST `/student/{studentId}`, DELETE, PATCH, PUT, cele 3 GET-uri) sunt fără `@PreAuthorize` ; orice autentificat adaugă, șterge sau modifică cărțile oricui.
- Corecte: `CourseController.java:28–33` și `:35–40` (POST/DELETE cu `COURSE_MANAGE`).
- Endpoint-uri moarte, nu vulnerabilități: `StudentController.java:27–32` (POST `/add`, `USER_ADD`), `:34–39` (DELETE, `USER_DELETE`), la fel `ProfessorController.java:35–37` și `EnrolmentController.java:36–41` ; permisiunile nu sunt acordate nimănui (vezi M3).

### B3 🔴 `updateCourse` : fără id în body dă 500, cu id în body inserează 2 cursuri noi

`src/main/java/ro/mycode/sebischool/course/service/commandService/CourseCommandServiceImpl.java:53`

**CUM E ACUM:** `PUT /courses/1 {"name":"Spring Boot","departament":"CS"}` -> `500` ( `IllegalArgumentException: The given id must not be null` ). Același PUT cu `"id":1` în body -> `200` cu `{"id":3...}` ; `GET /courses` arată 3 cursuri „Spring Boot" (id 1, 2, 3), iar 2 și 3 au `professor_id = null`.

**DE CE:** Linia 53 caută cursul după `courseRequest.getId()`, nu după id-ul din path. Câmpul din body e aproape mereu `null`, iar `findById(null)` aruncă `IllegalArgumentException`. Pe aceasta n-o prinde niciun `@ExceptionHandler`, așa că iese 500.

Când trimiți totuși id-ul, apar trei efecte în același timp. `setName` / `setDepartament` pe `course`, care e entitate gestionată, modifică rândul 1 prin dirty checking la commit, fără să apelezi `save`. Apoi `save(CourseMapper.toEntity(...))` se apelează de DOUĂ ori, cu obiecte NOI fără id, deci fiecare face un INSERT: rândurile 2 și 3. Din cauza asta un nume dublat trece fără `existsByName`. Răspunsul primit e al copiei 3, nu al cursului pe care l-ai editat. Testul tău `CourseIntegrationTest.updateCourse_persistsChange` e `@Disabled` exact pe acest bug.

**FIX:**

```
Course course = courseRepository.findById(id).orElseThrow(CourseNotFoundException::new);
course.setName(..);
course.setDepartament(..);
return CourseMapper.toDto(course);
```

Fără niciun `toEntity` și fără `save` dublu. Scoți apoi `@Disabled` de pe test, care trebuie să treacă.

**Același bug la:**

- `CourseCommandServiceImpl.java:58` - `save(CourseMapper.toEntity(courseRequest))` fără id inserează un curs nou, iar rezultatul e aruncat.
- `CourseCommandServiceImpl.java:59` - același apel în `return`: al doilea insert. Liniile 56-57 modifică `course`, dar valorile se pierd din răspuns, pentru că DTO-ul se construiește din entitatea nouă.
- `CourseCommandServiceImpl.java:31` - `addCourse` folosește tot id-ul din body: `enrolmentRepository.countByCourseId(courseRequest.getId())`. La creare id-ul e `null`, deci verificarea `CourseFullException` (`> 2`) numără înscrieri pentru un curs care nu există încă și nu se declanșează niciodată (din cod nu pot confirma ce întoarce interogarea derivată cu parametru null).
- `course/dtos/CourseRequest.java:14` - `Long id` în DTO-ul de request îi permite clientului să trimită id-ul; cauza structurală. Id-ul trebuie să vină doar din path.
- `CourseController.java:51–53` - controllerul pasează `@PathVariable Long id`, dar service-ul îl ignoră (`CourseCommandServiceImpl.java:52`).
- Curate (verificate): `BookCommandServiceImpl.java:45, :62`, `StudentCommandServiceImpl.java:37, :66`, `ProfessorCommandServiceImpl.java:44, :60`, `EnrolmentCommandServiceImpl.java:58, :84`, `CourseCommandServiceImpl.java:65` - toate folosesc id-ul din path. `BookCommandServiceImpl.java:36–38` (`addBook`) face `save(toEntity)` fără id, corect la creare.

### B4 🔴 `deleteProfessor`: `NullPointerException` pe câmpul `course` (null)

`src/main/java/ro/mycode/sebischool/professor/service/ProfessorCommandServiceImpl.java:77`

**CUM E ACUM:** `DELETE /profesori/1` (cu `USER_DELETE` dat manual în H2) -> `500`. Log: `NullPointerException: Cannot invoke "Course.setProfessor(...)" because "this.course" is null`.

**DE CE:** La linia 25 ai un CÂMP de instanță `Course course;`. Nimeni nu-l setează vreodată, deci rămâne `null` pentru toată viața bean-ului. La linia 76, `professor.getCourses()` întoarce colecția, dar rezultatul nu e folosit nicăieri, așa că apelul nu face nimic. La linia 77, `course.setProfessor(null)` se apelează pe câmpul null, nu pe cursurile profesorului, și de aici NPE-ul. Intenția era să desfaci legătura curs -> profesor înainte de DELETE, ca foreign key-ul `professor_id` să nu blocheze ștergerea. Pentru asta trebuie să iterezi colecția. Un câmp într-un `@Component` singleton e, oricum, stare comună tuturor cererilor.

Eroarea a apărut abia după ce am dat manual `USER_DELETE` în baza de date, pentru că niciun cont nu primește această permisiune la register (vezi M3).

**FIX:** ștergi câmpul `Course course;`, apoi:

```
professor.getCourses().forEach(c -> c.setProfessor(null));
professorRepository.delete(professor);
```

**Același bug la:**

- `ProfessorCommandServiceImpl.java:25` - câmpul `Course course;` e cauza de fond; oricine îl folosește în alte metode primește același null.

- `:76` - `professor.getCourses();` e un apel fără efect. Dacă `Professor.courses` e null, apelul însuși poate da NPE (`Professor.java:45` nu inițializează colecția).
- `:80` - `professorRepository.delete(professor)` rulează fără ca cursurile să fie detașate; din cauza lipsei de cascade pe `Professor.java:44` și a FK-ului `course.professor_id`, un profesor cu cursuri ar încălca FK-ul chiar după ce repari NPE-ul de la `:77`.
- `:92–93` și `:107–108` (`assignCourse`, `unassignCourse`) folosesc corect variabila locală, dar `professor.getCourses().add/remove` presupune că Set-ul nu e null; `Professor.courses` nu are inițializare.
- Restul serviciilor sunt curate: câmpurile `XRepository x;` din celelalte servicii sunt atribuite prin constructor, iar celelalte delete-uri au cascade ALL + `orphanRemoval` (`CourseCommandServiceImpl:46`, `StudentCommandServiceImpl:60`, `BookCommandServiceImpl:56`, `EnrolmentCommandServiceImpl:77`).

### B5 🔴 PATCH student acceptă un email deja folosit: ambele conturi nu se mai pot loga

`src/main/java/ro/mycode/sebischool/student/service/commandService/StudentCommandServiceImpl.java:77`

**CUM E ACUM:** Ana face `PATCH /students/2 {"email":"ana@x.ro"}` -> `200`. Tabela `user` are acum doi utilizatori cu `ana@x.ro`. Login `ana@x.ro` -> `401`, login `bob@x.ro` -> `401`.

**DE CE:** La register verifici `findByEmail(...).isPresent()`, dar la PATCH (și la PUT, linia 47) setezi emailul direct pe `User`, fără nicio verificare. Nici coloana nu e `unique` în DB. La login, `DaoAuthenticationProvider` apelează `loadUserByUsername` -> `findByEmail`, care întoarce `Optional<User>`. Când query-ul găsește 2 rânduri, Spring Data aruncă `IncorrectResultSizeDataAccessException`. Security o împachetează în `InternalAuthenticationServiceException` (asta apare în log), iar `ExceptionTranslationFilter` o tratează ca pe orice `AuthenticationException` și răspunde cu `401` prin entry point-ul tău. Ana își pierde contul, iar Bob nu se mai poate loga cu emailul vechi, pentru că emailul lui nu mai există.

**FIX:** în `updatePatchStudent` și `updateStudent`: dacă emailul nou diferă de cel actual și `userRepository.findByEmail(nou).isPresent()`, arunci `EmailAlreadyExistsException` (deja mapată pe 409). Plus `@Column(unique = true)` pe `User.email`, ca DB-ul să fie a doua plasă.

**Același bug la:**

- `StudentCommandServiceImpl.java:47` - PUT `/students/{id}`: `u.setEmail(...)` fără verificare de duplicat (citit în cod, nereprodus).
- `users/model/User.java:32` - `email` fără `@Column(unique = true)`; cauza de fond. `ddl-auto: update` nu adaugă ulterior constrângerea pe date deja duplicate.
- `auth/authService/AuthServiceImpl.java:76` - register verifică unicitatea cu `findByEmail(...).isPresent()` (check-then-act, fără constrângere în DB); sub cereri concurente se pot crea 2 useri cu același email.
- `student/repository/StudentRepository.java:28` - `existsByUserEmail(String)` există și nu e folosit nicăieri. La PATCH/PUT trebuie exclus userul curent (emailul propriu nemodificat nu e duplicat), deci ai nevoie de `existsByEmailAndIdNot` sau echivalent pe `UserRepository`.
- Nu e afectat: `ProfessorCommandServiceImpl.java:42–70` (nu modifică emailul). `UserMapper.java:16` pune email, dar n-am confirmat că e apelat într-un flux care scrie în DB.

### B6 🔴 Lipsește `@Valid` pe PUT/PATCH profesor: date invalide dau 500 în loc de 400

`src/main/java/ro/mycode/sebischool/professor/controller/ProfessorController.java:44`

**CUM E ACUM:** `PUT /profesori/1` cu `"departament":""` -> `500` ; `PATCH /profesori/1` cu `{}` -> `500` ; PUT cu `yearExperience` -7 -> `500` . Log: `ConstraintViolationException ... rootBeanClass=Professor ... during update time` .

**DE CE:** Fără `@Valid` pe `@RequestBody` , Spring nu citește deloc `@NotBlank` din `ProfessorRequest` și `ProfessorPatchRequest` , iar cererea ajunge în service cu date goale. Ce vezi tu e a doua plasă: Hibernate revalidează ENTITATEA `Professor` (care are `@NotBlank` și `@Min(0)` ) chiar înainte de UPDATE, la commit. De-aia mesajul spune `during update time` și de-aia primești 500 ( `TransactionSystemException` ), nu 400: eroarea apare după ce cererea a trecut deja de controller, într-un loc pe care `GlobalExceptionsHandler` nu-l acoperă.

**FIX:** `@Valid @RequestBody` pe `updateProfessor` (linia 44) și `updatePatch` (linia 51). Pe `ProfessorPatchRequest` , câmpul opțional nu mai are `@NotBlank` (un PATCH gol e legitim), iar în service setezi `specialty` doar dacă e `!= null` . Plus `@Min(0)` pe `ProfessorRequest.yearExperience` .

**Același bug la:**

- `ProfessorController.java:51` - `updatePatch` fără `@Valid` (reprodus cu `{}` și `{"specialty":""}` ).
- `CourseController.java:44` - `patchCourse` fără `@Valid` . Reprodus: `PATCH /courses/{id}` cu `{"name":""}` dă `ConstraintViolationException` (500); cu `{"name":"ab"}` întoarce `200` și salvează „ab", deși `CoursePatchRequest` are `@Size(min=3,max=50)` . Aceeași problemă pentru `departament` .
- `EnrolmentController.java:44` - `patchEnrolment` fără `@Valid` . Nu am reprodus un 500 ( `EnrolmentPatchRequest` nu are constrângeri), dar încalcă aceeași regulă și nu se apără dacă adaugi constrângeri.
- Cauză comună: `system/exceptions/GlobalExceptionsHandler.java` nu are handler pentru `MethodArgumentNotValidException` sau `ConstraintViolationException` . Chiar și după ce pui `@Valid` peste tot, erorile de pe controllerele care îl au deja ( `CourseController:30,51` ; `StudentController:29,42,49` ; `BookController:29,43,50` ; `EnrolmentController:31,50` ; `AuthController:28,33` ) ies cu corpul implicit Spring, nu cu `ApiErrorResponse` -ul aplicației (vezi M10).
- Controllere corecte: Student, Book, Course POST/PUT, Enrolment POST/PUT, Auth. Încălcările sunt doar cele 4 metode de mai sus.

---

## 🟡 Importante

- **M1** `ProfessorController.java:57` : `GET /profesori` cere `USER_EDIT` , ar trebui `USER_VIEW` (C3 din iunie, neatins).
- **M2** `ProfessorController` : lipsește `GET /profesori/{id}` ; serviciul `getProfessorById` există, dar `GET /profesori/1` -> 405 (C2 din iunie, pe jumătate).
- **M3** `AuthServiceImpl.java:135` : nimeni nu primește `USER_DELETE` sau `USER_ADD` , deci DELETE pe `/students` , `/enrolments` , `/profesori` și POST `/students/add` sunt imposibil de apelat.

- **M4** `AuthServiceImpl.java:91` : `userType` vine din cererea publică de register, deci oricine își creează singur cont PROFESSOR cu `COURSE_MANAGE` .
- **M5** `ProfessorRequest.java:9–11` : `firstName` / `lastName` sunt `@NotBlank` , dar `updateProfessor` le ignoră; clientul crede că a schimbat numele.
- **M6** `ProfessorPatchRequest` : tot doar `specialty` (H2 din iunie, neatins).
- **M7** `CourseCommandServiceImpl.java:31` : `CourseFullException` se verifică la CREAREA cursului, cu `countByCourseId(null)` ; la `addEnrolment` limita nu se verifică deloc.
- **M8** `Professor.java:52–57` : `equals` / `hashCode` includ `user` și `courses` (lazy), deci risc de ciclu și de query-uri ascunse (M2 din iunie, neatins).
- **M9** `pom.xml:23` : Lombok 1.18.30 nu compilează pe JDK 25 (default-ul de pe Mac: `TypeTag :: UNKNOWN` ); merge doar pe 17; urcă la 1.18.38+.
- **M10** `GlobalExceptionsHandler` : nu tratează `MethodArgumentNotValidException` , `IllegalArgumentException` și `UnsupportedOperationException` , deci acestea ies ca 500 generic.

## 🟢 Cleanups

- **C1** `ProfessorCommandServiceImpl` / `EnrolmentQueryServiceImpl` / `CourseQueryServiceImpl` : `@Component` în loc de `@Service` (N4 din iunie).
- **C2** `StudentSummaryResponse.java:10` : `@Service` pe un DTO, deci Spring creează un bean inutil.
- **C3** `Professor.java:39` : cascade ALL de la Professor la User (șterge contul la delete); ia o decizie explicită.
- **C4** `ProfessorMapper` : `@Component` + metode statice, adnotarea e inutilă.
- **C5** Typo-uri: `SecurityAcessDeniedHandler` , `JWTAuthentificationEntryPoint` , `Bookrequest` ; `HintsConstants` nefolosit.
- **C6** `CourseRepository.java:25` : `List<Course> id(Long id)` e o metodă rămasă din greșeală.
- **C7** `EnrolmentController.java:62` : metoda `getEnrolmentsByStudent` primește de fapt `courseId` .

---

## Before / After pentru 🔴

| | Before (acum) | After (corect) |
|---|---|---|
| B1 | `Set<Course> courses` în `ProfessorResponse` | `List<CourseSummaryResponse> courses` (id, name, departament), mapat în `ProfessorMapper` |
| B2 | `@PreAuthorize("hasAuthority('USER_VIEW')")` pe PATCH; studentul are `USER_EDIT` | `hasAuthority('USER_EDIT')` pe PATCH; `USER_EDIT` doar la profesor (sau `PROFESSOR_EDIT`) + `@PreAuthorize` pe toate rutele de scriere |
| B3 | `findById(courseRequest.getId())` + `save(toEntity(..))` de două ori | `findById(id)` din path, `setName/setDepartament`, fără `save(toEntity)` |
| B4 | `course.setProfessor(null)` pe câmpul `Course course` (null) | `professor.getCourses().forEach(c -> c.setProfessor(null));` și câmpul șters |
| B5 | `u.setEmail(request.getEmail())` direct | verificare `findByEmail(nou)` -> `EmailAlreadyExistsException` + `@Column(unique = true)` |
| B6 | `@RequestBody ProfessorRequest request` | `@Valid @RequestBody ProfessorRequest request` |

## Q&A

1. La `GET /profesori` ai primit status `200`, dar JSON-ul era stricat și log-ul spunea „response committed already". De ce nu a mai putut Spring să schimbe statusul în `500` când Jackson a dat de `StackOverflowError`?
2. Ai `@NotBlank` și pe `ProfessorRequest` (DTO), și pe entitatea `Professor`. Care dintre ele te-a oprit când ai trimis `departament` gol, în ce moment al cererii și de ce rezultatul a fost `500`, nu `400`?
3. Ana are în token `USER_VIEW` și `USER_EDIT`. Ce verifică de fapt `hasAuthority('USER_EDIT')` și ce ar trebui să mai știe aplicația ca să o oprească pe Ana să modifice profesorul sau contul lui Bob?

---

## Runda anterioară — iunie 2026 (fără SHA, branch `feat/implement-professor-feature`)

Verificată prin rulare pe `de3524b`:

- ✅ C1 PATCH = PUT: închis ( `ProfessorPatchRequest` + `patchProfessor` ), dar a apărut B2 (permisiune `USER_VIEW` )
- 🟡 C2 `GET /{id}` : pe jumătate (serviciul există, endpoint-ul lipsește -> 405), vezi M2
- 🔴 C3 `GET /` pe `USER_EDIT` : NEATINS (M1)
- 🔴 C4 `@Valid` lipsă: NEATINS (B6, reprodus 500)
- ✅ C5 `addProfessor` -> `UnsupportedOperationException` : închis
- ✅ C6 `firstName` / `lastName` scoase de pe `Professor` + `@JoinColumn(unique, nullable=false)` : închis
- ✅ H1 `yearExperience` în update: închis
- 🔴 H2 `PatchRequest` doar `specialty` : NEATINS (M6)
- ✅ H3 `yearExperience` în mapper: închis
- ✅ H7 teste: închis (59 verzi, 1 `@Disabled` pe bug-ul B3)
- 🔴 M1 `deleteProfessor` : REDESCHIS, acum NPE (B4)
- 🔴 M2 `equals` cu relații lazy: NEATINS (M8)
- 🔴 N4 `@Component` : NEATINS (C1)

---

MyCodeSchool Material didactic MyCodeSchool **mycodeschool.ro**